

1inch swap-vm Fee.sol Security Review

: 1inch SwapVM Aqua protocol fee best-effort collection

Jul 28, 2026

Revision 1.0

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

1inch swap-vm Fee.sol Security Review	1
Table of Contents	2
Executive Summary	4
Audit Overview	8
Scope	8
Code Revision	8
Severity Categories	9
Status Categories	10
Finding Breakdown by Severity	11
Findings	13
Summary	13
Revision History	14

Executive Summary

Beginning on July 28, 2026, ChainLight of Theori conducted a 1-day differential security review of 1inch `swap-vm` (v1.0.1 → v1.0.2), focused on the `amountIn` Aqua protocol-fee best-effort change in `Fee.sol` and its settlement accounting impact.

The scope is `src/instructions/Fee.sol`. `AmountIn` Aqua protocol fee collection moved from strict revert on uncoverable fee to best-effort collection (`_tryPullFee`, `ProtocolFeeSkipped`, success-only `amountNetPulled`), with a hard revert for zero fee recipient when fee bps is non-zero.

Reportable issues: 0. No settlement break, fund theft, or taker underpayment under scope. Soft-fail is limited to fee `Aqua.pull`. Wallet-charging protocol fees, settlement transfers, dynamic fee-provider validation errors, and zero-recipient checks remain hard-fail. Uncollected fee on skip is accepted product risk: pricing stays fee-inflated, residual stays with the maker; operators should monitor `ProtocolFeeSkipped`.

Threat Model / Scenarios

- **Assets:** protocol fee revenue; maker Aqua ledger and related inventory used for fee collection; taker `amountIn`; `amountNetPulled` at pre-push; order liveness when the fee cannot be covered.
- **Adversaries:** maker under-funds fee inventory or revokes Aqua allowance; token refuses transfer to the fee recipient; malicious or faulty dynamic fee provider.
- **Primary objectives:** keep settlement hard-fail correct; ensure a skipped fee does not reduce taker payment; confine fail-open behavior to fee collection; keep zero-recipient, wallet-charging fee, and provider-error paths hard-fail.
- **Representative scenarios:**
 - uncoverable Aqua fee pull → swap proceeds, `ProtocolFeeSkipped`, residual with maker
 - successful fee pull → `amountNetPulled` increases by the fee
 - skip with success-only `amountNetPulled` → taker payment is not reduced by the skipped fee
 - zero fee recipient with non-zero bps → hard revert
 - wallet-charging protocol fee shortfall → hard revert (SafeERC20)
 - token blocks fee recipient → non-collection without maker collusion
 - dynamic fee provider failure or bps out of range → hard revert
 - Aqua fee opcode without fundable Aqua inventory (e.g. signature-mode, no Aqua ship) → permanent soft skip of fee

Trust Boundaries

- **Untrusted:** ERC-20 transfer behavior (blacklist, false returns); maker inventory and Aqua allowance; dynamic fee-provider data and bps; maker programs that include fee opcodes.
- **Untrusted (execution-level):** `Aqua.pull` failures during fee collection (ledger short, amount bounds, transfer failure) → soft only inside `_tryPullFee`.
- **Trusted (as implemented):** Fee soft boundary as coded; SwapVM settlement outside the fee try/catch; zero-recipient and wallet-charging fee hard guards.
- **Operational trust:** validate and whitelist strategies before exposure; pin fee recipient and fee rate; use Aqua fee opcodes only on Aqua-mode orders; monitor `ProtocolFeeSkipped`.

Assumptions

- Best-effort amountIn Aqua fee is intentional: prefer order liveness over guaranteed fee collection when the maker cannot cover the fee from pre-existing Aqua inventory.
- On skip, pricing stays fee-inflated; uncollected fee stays with the maker. Documented accepted risk.
- Fee pull runs in the instruction loop before taker→maker credit of `tokenIn`. Fee is not funded from same-swap taker credit.
- Wallet-charging protocol fee opcodes remain strict and are outside the best-effort catch.
- Standard ERC-20 is the intended token class. Non-standard tokens are external risk at the transfer boundary.

Discussion

- **Best-effort fee collection:** Soft-fail applies only to amountIn Aqua fee pull. On failure the swap can succeed, protocol fee may be zero, and residual stays with the maker. Accepted risk. Monitor `ProtocolFeeSkipped`.
- **Maker permanent non-collection:** Maker can prevent collection via thin or zero Aqua inventory or revoked allowance. Same residual-to-maker model as skip. Accepted risk for protocol revenue; not a settlement integrity break.
- **Token permanent non-collection:** Token can refuse transfer to the fee recipient (blacklist or false return) without maker collusion. Soft skip path. Ops and token-integration risk; not a settlement integrity break.
- **Exception boundary:** Soft-fail is limited to fee `Aqua.pull` in `_tryPullFee`. Settlement, wallet-charging fees, and dynamic provider validation errors stay outside the catch. Expansion of the catch to free settlement or unrestricted drain does not hold.

- **Success-only `amountNetPulled`** : Credit only after successful fee pull. Taker underpayment via skipped fee does not occur.
- **Zero recipient**: Zero recipient with non-zero bps hard-reverts in this release. Silent permanent free fee via zero recipient does not occur.
- **Wallet-charging protocol fee**: Remains strict (SafeERC20). Best-effort collection does not hold on this path.
- **Never-pay config residual**: Zero recipient hard-reverts, but other never-pay setups (e.g. Aqua fee opcode on signature-mode without Aqua inventory) soft-skip every fill. Program bytes may still present as fee-paying while protocol revenue is zero. Mitigate off-chain via strategy validation, whitelist, and fee recipient pin (as documented operational requirements).

Summary of Findings

The audit revealed a total **0** issues, categorized by severity as follows:

- **Critical**: 0
- **High**: 0
- **Medium**: 0
- **Low**: 0
- **Informational**: 0

Audit Overview

Scope

Name	1inch swap-vm Fee.sol Security Review
Target / Version	SwapVM (differential): https://github.com/1inch/swap-vm • Baseline: tag <code>v1.0.1</code> • Target: tag <code>v1.0.2</code> (https://github.com/1inch/swap-vm/releases/tag/v1.0.2) • Compare: https://github.com/1inch/swap-vm/compare/v1.0.1...v1.0.2 • Scope: <code>src/instructions/Fee.sol</code> Deployment: The reviewed change is included in the latest release: • Release: https://github.com/1inch/swap-vm/releases/tag/v1.0.2 Out of scope: • Full SwapVM instruction set, routers, hooks, extruction redesign • Aqua core rewrite beyond fee-pull interface behavior • Wallet-charging fee redesign (hard-fail control check only) • Mainnet exploit • Oracles, fee providers, tokens beyond interface behavior for this change • Deployment, ops tooling, off-chain aggregator code
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	• N/A
High	0	• N/A
Medium	0	• N/A
Low	0	• N/A
Informational	0	• N/A
Note	0	• N/A

Findings

Summary

#	ID	Title	Severity	Status
---	----	-------	----------	--------

Revision History

Version	Date	Description
1.0	Jul 28, 2026	Initial version

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

